WAREHOUSE ROBOT NAVIGATION SYSTEM

(Working Duration: Monday of Week 10 to Monday of Week 14 – 30 Marks)

STORY BACKGROUND:

A large e-commerce warehouse is adopting autonomous robots to improve efficiency in handling customer orders. These robots are responsible for locating items, picking them from shelves, and delivering them to packing stations. The warehouse is divided into multiple zones, with thousands of items stored across different shelves. Every day, a high number of orders are received, requiring fast and accurate coordination between robots.

To ensure smooth operations, the system must:

1. Manage incoming orders efficiently
2. Assign tasks fairly among robots
3. Help robots navigate through the warehouse
4. Quickly locate items
5. Allow robots to return safely after completing tasks

To achieve this, the system is designed using several fundamental data organization techniques. Incoming orders are handled in a structured sequence to ensure fair processing. Robot assignments follow a rotating mechanism so that workload is evenly distributed. The warehouse layout is modeled in a hierarchical form to represent zones, aisles, and shelves, enabling efficient navigation and path planning. Additionally, robot movements are recorded step-by-step, allowing the system to retrace and reverse the path taken. This ensures that once a task is completed, the robot can return to its starting point safely and efficiently.

One special feature of the system is the ability for robots to trace back their movement path and return using the exact reverse route, ensuring efficiency and avoiding navigation errors. Overall, the system is developed by applying key concepts such as ***stack, queue, circular queue, and tree structures*** to ensure efficiency, fairness, and accuracy in warehouse operations.

FULL SYSTEM WORKFLOW:

All modules must work together as one complete system. The full system flow as below:

1. A new order is received
2. The order is stored and prepared for processing
3. A robot is assigned to the task
4. The system identifies the item location
5. A route is generated for the robot
6. The robot moves step-by-step to the item
7. After completing the task, the robot returns using the reverse path

System Tasks That Can Be Allocated to Each Member for This Assignment

Managing a warehouse robot navigation system requires coordinating multiple complex operations to ensure smooth and efficient order fulfilment. Each component of the system involves specific tasks that can be optimized using suitable data structures such as ***Stack, Queue, Circular Queue, and Tree structures***.

The system is ***not limited to the following modules***; ***however,*** the tasks outlined below represent the ***core functionalities*** essential for the successful operation of the warehouse system:

TASK 1: ORDER MANAGEMENT MODULE

The Order Management Module is responsible for handling all incoming customer orders within the warehouse. It ensures that every order is recorded, organized, and processed in a structured and fair manner based on its arrival time.

Functional Requirements

- Accept and record new customer orders into the system
- Maintain an ordered list of all incoming requests
- Process orders sequentially according to their arrival
- Remove orders from the list once they are assigned to robots
- Display current pending and completed orders
- Handle exceptional cases such as empty order lists or system overload

Key Features:

- Ensures fair processing of orders
- Maintains real-time order status updates
- Supports continuous inflow of new orders

Expected Core Output:

- List of pending orders
- Current order being processed
- Completed order history

TASK 2: ROBOT ASSIGNMENT MODULE

The Robot Assignment Module manages how tasks are distributed among available robots. It ensures that all robots are utilized efficiently and fairly by assigning tasks in a rotating and balanced manner.

Functional Requirements

- Maintain a list of all robots and their current status (available/busy)
- Assign tasks to robots in a continuous rotation
- Skip robots that are currently unavailable or under maintenance
- Track task assignments for each robot
- Ensure uninterrupted task allocation without restarting the assignment cycle

Key Features:

- Balanced workload distribution among robots
- Continuous and automated assignment process
- Real-time robot availability tracking

Expected Core Output:

- Robot assignment list
- Current robot handling each task
- Robot status overview

TASK 3: ROBOT NAVIGATION AND PATH TRACKING MODULE

This module controls the movement of robots within the warehouse and ensures accurate navigation. It also enables robots to return safely by retracing their movement path in reverse order.

Functional Requirements

- Record each movement step taken by the robot (e.g., forward, left, right)
- Store the full path from starting point to destination
- Allow the robot to reverse its path step-by-step after completing a task
- Handle navigation issues such as obstacles or incorrect paths
- Simulate robot movement through logs or step-by-step visualization

Key Features:

- Accurate path tracking and movement recording
- Reverse navigation capability for return journeys
- Basic obstacle handling through backtracking

Expected Core Output:

- Forward movement path
- Reverse path for returning
- Complete navigation log

TASK 4: ITEM SEARCH AND MANAGEMENT MODULE (OPTIONAL)

The Item Search and Management Module is responsible for storing and managing all item-related information in the warehouse. It allows quick and efficient retrieval of item locations when required.

Functional Requirements

- Store item details such as Item ID, name, and location
- Insert new items into the system
- Search for items based on specific criteria (e.g., ID or name)
- Update or delete item records when necessary
- Display items in a structured and sorted format

Key Features:

- Fast and efficient item lookup
- Organized storage of item information
- Easy management of item records

Expected Core Output:

- Search results for requested items
- Updated item database
- Organized item listing

TASK 5: WAREHOUSE LAYOUT AND NAVIGATION MODULE (OPTIONAL)

This module represents the physical structure of the warehouse and supports navigation by defining relationships between zones, aisles, and shelves.

Functional Requirements

- Model the warehouse layout into structured sections (zones, aisles, shelves)
- Define connections between different locations
- Provide navigation routes from one point to another
- Support traversal through all warehouse sections
- Integrate with the robot navigation module for path planning

Key Features:

- Structured representation of warehouse layout
- Efficient route generation
- Scalable design for large warehouse environments

Expected Core Output:

- Visual or textual representation of warehouse layout
- Path between selected locations
- Traversal results

Lab Work #2 – Program & Live Presentation Guidelines (30 Marks)

1. A team can contain a minimum of **THREE (3)** members and a maximum of **FIVE (5)** members.

   Minimum 3 members: Each member will be responsible for one of the mandatory modules:
   1. Order Management Module
   2. Robot Assignment Module
   3. Robot Navigation and Path Tracking Module

   Optional 2 members: If the team has **four or five** members, the additional members can handle the optional modules:
   4. Item Search and Management Module
   5. Warehouse Layout and Navigation Module

2. Each team member is required to use at least **ONE (1)** suitable data structure along with its appropriate algorithms to carry out their assigned task. *Marks will be awarded based on individual contributions, taking into account each member's responsibility within the system and the accuracy and justification of their choice of data structures and algorithms.*

3. Your team is required to use C++ programming to develop **ONLY ONE (1)** prototype in this section.

4. **Built-in containers such as <list>, <vector>, etc. are not allowed** in this assignment. All containers are self-created.

   Refer to the link: https://www.geeksforgeeks.org/containers-cpp-stl/ for further information on built-in containers in STL C++.

5. The evaluation criteria for this lab work #2 also include assessing the clarity and structural design of the code, as well as the quality of comments and adherence to good programming practices. (e.g., indentation, meaningful identifier names, comments, etc.).

6. **This task requires a single group submission; however, grading will be based on the individual contributions of each team member to the system.**

   The team leader must upload a ZIP file of the system solution to the Moodle system by **Monday of Week 14**, (_____), no later than 5:00 pm.

   The zip file must adhere to the following name format:

   "<GroupNo>_<student ID-leader>_<student ID-member1>_<student ID-member2>_<student ID-member2>.zip"

   For example, **"G1_TP012345_TP012344_TP012123_TP012111.zip"**

   Refer to **Page 6** for marking criteria of this Lab Evaluation Work #2 submission.

7. After submitting your system code to Moodle, your team must schedule ***a live presentation*** with your lecturer *between Week 14 to Week 16 (Your final exam week #1)*.

Summary: What Do You Need to Hand in During this Assignment Submission?

1. This assignment requires **TWO (2)** submissions by your team, which include the following:

   i. Lab Work #2 – Group Submission
   - Submit your solution by uploading the *.cpp, .hpp, and CSV/text* files individually to Moodle. Do not upload them as a ZIP folder.

   ii. Individual Live Demonstration (30 Marks)
   - Each member must present their specific contribution to the system.
   - The presentation should be completed within 30 minutes, including both the **system demonstration and a Q&A session**.
   - PowerPoint slides are not required for this demonstration.

2. Your team will need to submit all your C++ solutions to the Moodle system by **Monday of Week 14, (_____) before 5:00 pm**, and arrange *a live presentation with your lecturer between Tuesday of Week 14 and Friday of Week 16*.

3. **Missing the live presentation** will result in your assignment task #2 receiving **0 marks**.

AI USAGE POLICY FOR CODING ASSIGNMENTS

1. **Students are strictly prohibited** from using artificial intelligence (AI) tools to generate complete or partial solutions, source code, algorithms, or program logic for this assignment.

2. The use of AI tools is permitted **only for supportive purposes**, such as:
   - Understanding programming concepts or syntax
   - Identifying relevant keywords or technical terms
   - Clarifying error messages or documentation terminology

3. **All submitted code must be the student's own original work.** Lecturers reserve the right to conduct oral questioning, code tracing, or live demonstrations to verify the authenticity and understanding of submitted work.

4. Therefore, students must be able to *explain and justify every part of their code* during evaluation or questioning.

5. If AI tools are **used for assistance,** students *must clearly declare the purpose of AI usage* e.g., "AI was used to identify appropriate keywords related to file handling in Python"

6. **Copying, paraphrasing, or adapting AI-generated code or solutions**, whether modified or unmodified, is **considered a violation of academic integrity.**

7. **Any misuse of AI tools**, including generating solutions beyond permitted assistance, **may result in penalties** including:
   - **Zero (0)** marks for the assignment **OR**
   - **Disciplinary action** in accordance with institutional policies

MARKING CRITERIA

(Lab Evaluation Work #2 - 30 MARKS)

This Lab Evaluation Work #2 will be assessed based on the following ***INDIVIDUAL*** performance criteria:

Assessment Components	Inclusive	30 Marks
CLO3: Lab Evaluation Work #2 – Individual Development Skills		
Practical Skills: Problem-Solving Skills (15 Marks)		
• Identify and address technical challenges.	Assessment of Problem-Solving Ability	
• Use of data structures and algorithms.	Technical Proficiency	
• Implementation of features according to design specifications.	Technical Proficiency	
• Code quality, including readability, efficiency, and correctness.	Code Quality Evaluation	
• Quality of individual contributions relative to team goals.	Contribution Assessment	
• Innovation and creativity in developing and implementing features.	Creativity and Innovation Evaluation	
Practical Skills: Q&A with Justification of Data Structures (15 Marks)		
• Clear and logical explanation for the choice of data structures.	Justification Ability	
• Relevance of chosen data structures to the functionality implemented.	Relevance Evaluation	
• Justification aligned with the system requirements and performance needs.	Alignment with Requirements	
• Effectiveness of the live presentation in explaining individual contributions	Presentation Effectiveness	